MLOps
(S1-25_AIMLCZG523)
ASSIGNMENT - I

MLOps Heart Disease Prediction – Project Report

Group No: 13

Assignment Group Members:

Name	BITS ID	Contribution
KOUSHIK SADHUKHAN	2024aa05797	100%
CHOCALINGAM L	2024ab05275	100%
GEETANJALI ANANTRAO UDGIRKAR	2024aa05800	100%
G PRANAVANATHAN	2024ab05277	100%
DURGA PRASAD YADAV	2024ab05147	100%

1. Project Overview

- This project implements an **end-to-end MLOps pipeline** for predicting heart disease using machine learning.
- It covers data analysis, model training, experiment tracking, containerization, CI/CD automation, and monitoring, following production-grade MLOps best practices.

2. Code Repository:

https://github.com/2024ab05275-chocs/mlops-heart-disease/tree/main

3. Setup & Installation Instructions

3.1 Tools and technologies Needed

3.2 Installation Instructions:

- **Instructions are automated** and available in below GITHub Path
    - https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/main/scripts/setup/run_scripts.sh

3.2.1 Python Environment Setup

Required Python Version

- **Python 3.10** is mandatory for this project.

Local Execution

If Python 3.10 is not found on the local machine,

the script installs it automatically:

- **macOS:** Uses Homebrew (brew install python@3.10)
- **Linux:** Uses APT package manager (apt install python3.10)

CI Execution

- Python installation is skipped
- The script fails early if Python 3.10 is unavailable, enforcing environment correctness

- **This approach guarantees version consistency across all environments.**

3.2.2 Package Manager Upgrade

Before installing dependencies, the script:

- Upgrades **pip** to the latest available version
- Prevents compatibility issues with modern Python packages

3.2.3. Dependency Installation

All project dependencies are installed using:

python3.10 -m pip install -r requirements.txt

mlops-heart-disease > requirements.txt

```
1   tabulate
2   mlflow>=2.0
3   pandas>=1.5
4   scikit-learn>=1.2
5   matplotlib>=3.6
6   seaborn>=0.12
7   numpy>=1.23
8   joblib==1.3.2
9   fastapi
10  uvicorn
11  pytest
12  ruff
13  prometheus-fastapi-instrumentator
```

The dependency list includes:

Core ML & Data Science Libraries

- **numpy** – numerical computing
- **pandas** – data manipulation
- **scikit-learn** – model training & evaluation
- **matplotlib, seaborn** – data visualization

Experiment Tracking

- **mlflow** – experiment tracking, metrics, and artifact logging

API & Model Serving

- **fastapi** – REST API framework
- **uvicorn** – ASGI server for model serving

Testing & Code Quality

- **pytest** – unit and integration testing
- **flake8** – code style enforcement

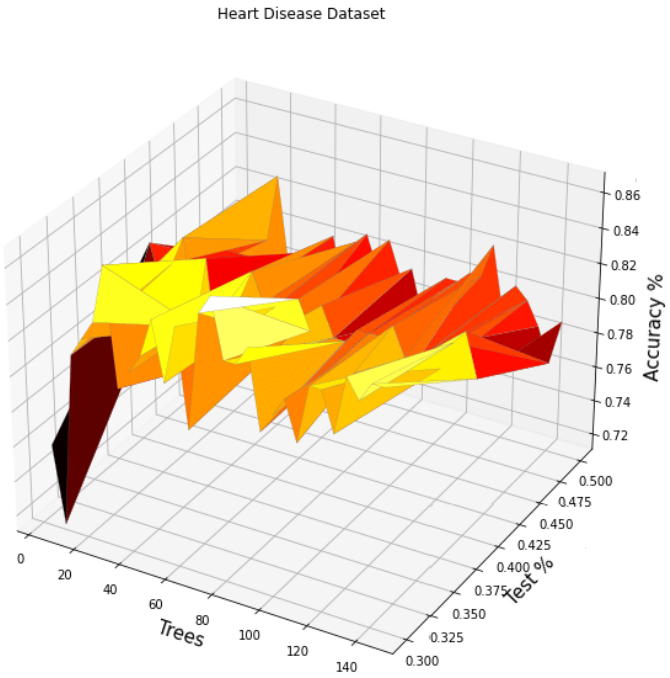
Monitoring

- **prometheus**-fastapi-instrumentator – exposes application metrics for Prometheus

4. Exploratory Data Analysis (EDA) & Modeling Choices

4.1 Dataset Overview

The project uses the **UCI Heart Disease dataset**, which contains clinical and demographic attributes used to predict the presence of heart disease.



Target Variable

- Original target values: `0,1,2,3,4`
  - `0` → No heart disease
  - `1–4` → Presence of heart disease
- **Transformation Applied:**
  - Converted into a **binary classification problem**
  - `target = 1` if original value > 0, else `0`

This decision simplifies the problem and aligns it with common medical risk classification use cases.

4.2. Exploratory Data Analysis (EDA)

EDA is implemented programmatically in **`preprocess.py`** to ensure reproducibility and CI compatibility.

- **/mlops-heart-disease/src/data/data_acquisition.py**
- **/mlops-heart-disease/src/data/preprocess.py**

4.2.1 Missing Value Handling

- Missing values in the raw dataset are represented using `?`
- These values are:
  - Identified during data loading
  - Replaced with `NaN`
  - Appropriately handled

during preprocessing

This ensures clean numerical input for downstream models.

4.2.2 Feature Inspection & Visualization

The following analyses are performed:

- Distribution plots for numerical features
- Correlation analysis using Seaborn heatmaps
- Tabular summaries using tabulate for readability

EDA visualizations are:

- Generated programmatically
- Saved as artifacts instead of being displayed
- Suitable for headless execution (CI/CD environments)

4.2.3 Feature Selection & Cleaning

- Redundant or irrelevant columns are excluded
- Data types are explicitly validated
- Processed datasets are stored under data/processed/ for reuse

This separation ensures raw data immutability and pipeline traceability.

4.3. Data Preprocessing Strategy

File Reference:
mlops-heart-disease/src/data/preprocess.py

4.3.1 Scaling

- **StandardScaler** is applied to numerical features
- Scaling is saved as a reusable artifact **(standard_scaler.pkl)**
- Ensures consistent transformation during inference

Scaling is necessary due to:

- Logistic Regression sensitivity to feature magnitude
- Improved convergence and stability

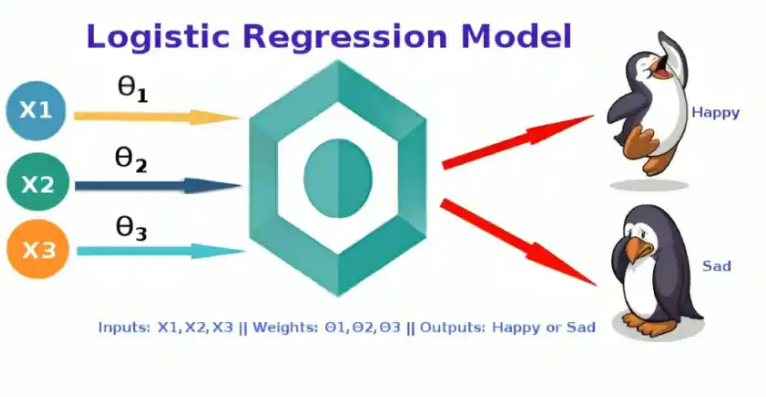
4.2.1 Train–Test Split

- Dataset is split into training and testing sets
- Ensures unbiased model evaluation
- Controlled through configuration for reproducibility

4.4. Modeling Choices

- Two models are implemented to balance **interpretability** and **performance**.
- **mlops-heart-disease/src/models/train_evaluate_logistic_regression.py**
- **mlops-heart-disease/src/models/train_evaluate_random_forest.py**

4.4.1 Logistic Regression (Baseline Model)



Why Logistic Regression?

- Interpretable coefficients
- Widely accepted in medical risk prediction
- Serves as a strong baseline

Implementation Details

- StandardScaler applied before training
- Cross-validation performed using cross_validate
- ROC curve and AUC metrics computed
- Headless plotting enabled using matplotlib.use("Agg")

Metrics Tracked

- ROC-AUC
- Cross-validation scores
- Model coefficients (implicitly via MLflow)

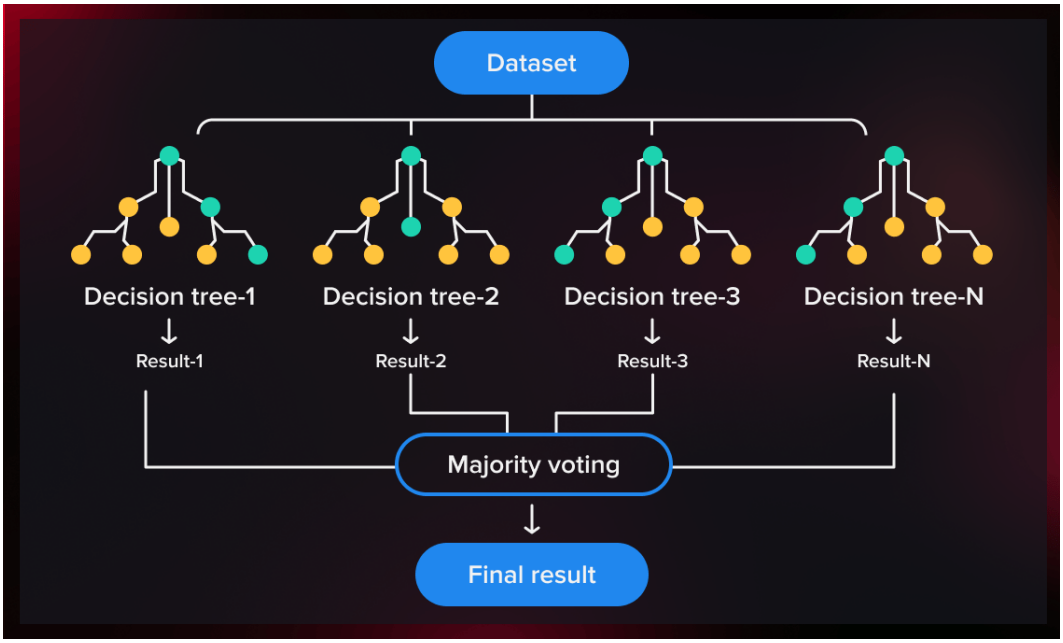
This model prioritizes transparency and explainability.

4.4.2 Random Forest Classifier

- Random Forest serves as the **performance-oriented model**, complementing Logistic Regression.

Why Random Forest?

- Captures non-linear feature interactions
- Robust to noise and outliers
- Performs well on tabular healthcare data



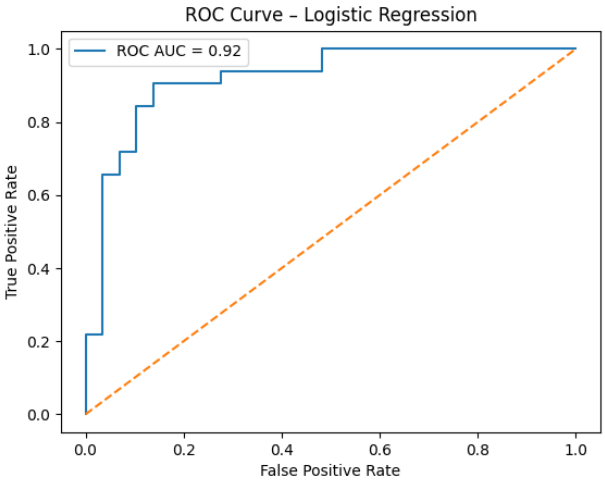
Implementation Details

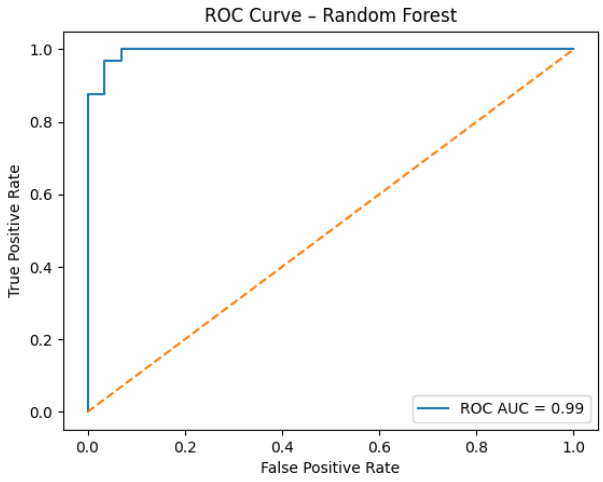
- Trained on preprocessed dataset
- Hyperparameters controlled via configuration
- Feature importance implicitly leveraged
- Evaluation includes ROC curve analysis

4.5. Model Evaluation Strategy

- **ROC curves** plotted and saved as artifacts
- **AUC scores** computed for both models
- **Cross-validation** used to assess generalization
- **All outputs are logged for traceability**

Plots and metrics are CI-safe and reproducible.





5. Experiment Tracking Integration

Both models integrate with **MLflow**:

What is tracked?

- Model Parameters
- Metrics (accuracy, precision, recall)
- Artifacts (plots, scalers, models)
- Experiment Metadata

Storage:

- Local SQLite database (`mlflow.db`)
- Artifact logging through MLflow's default backend

Benefits:

- Model comparison
- Reproducibility
- Version control for experiments

Design Rationale Summary

Decision	Justification
Binary target	Simplifies clinical interpretation
Programmatic EDA	CI/CD compatibility
StandardScaler	Required for linear models
Two-model approach	Balance interpretability & performance
Artifact saving	Production & deployment readiness

Experiment Tracking Summary

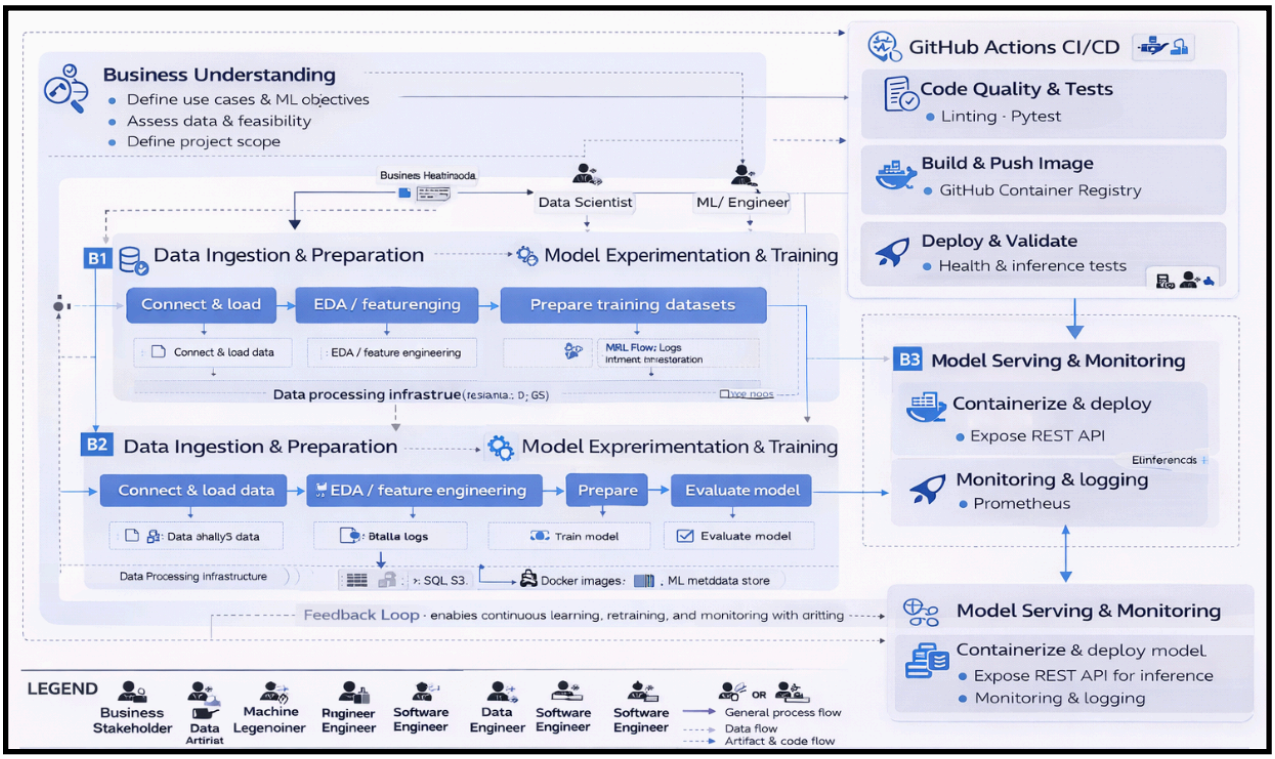
mlops-heart-disease/src/models/experiment_tracking.py

```
############################################################
# 3. Experiment Tracking
############################################################

$PYTHON -m src.models.experiment_tracking

############################################################
# 3.1 MLflow UI Server
############################################################

if [[ "$IS_CI" == false ]]; then
  echo "🚀 Starting MLflow UI server..."
  mlflow ui --host 0.0.0.0 --port 5000
else
  echo "⚙️ CI environment detected — MLflow UI not started"
fi
```

```
(base) chocalingamlakshmanan@Chocalingams-MacBook-Air MLOps-Assignment-sem3 % MLFLOW_WORKERS=1 mlflow ui \
  --backend-store-uri sqlite:///mlflow.db \
  --default-artifact-root ./mlruns \
  --host 127.0.0.1 \
  --port 7000
Registry store URI not provided. Using backend store URI.
2025/12/31 23:45:50 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2025/12/31 23:45:50 INFO mlflow.store.db.utils: Updating database tables
2025/12/31 23:45:50 INFO alembic.runtime.migration: Context impl SQLiteImpl.
2025/12/31 23:45:50 INFO alembic.runtime.migration: Will assume non-transactional DDL.
2025/12/31 23:45:50 INFO alembic.runtime.migration: Context impl SQLiteImpl.
2025/12/31 23:45:50 INFO alembic.runtime.migration: Will assume non-transactional DDL.
2025/12/31 23:45:50 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2025/12/31 23:45:50 INFO mlflow.store.db.utils: Updating database tables
2025/12/31 23:45:50 INFO alembic.runtime.migration: Context impl SQLiteImpl.
2025/12/31 23:45:50 INFO alembic.runtime.migration: Will assume non-transactional DDL.
[MLflow] Security middleware enabled with default settings (localhost-only). To allow connections from other hosts, use --host 0.0.0.0
 and configure --allowed-hosts and --cors-allowed-origins.
INFO:     Started server process [29421]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:7000 (Press CTRL+C to quit)
INFO:     127.0.0.1:51835 - "GET / HTTP/1.1" 200 OK
INFO:     127.0.0.1:51835 - "GET /static-files/static/js/main.bbdf6ce4.js HTTP/1.1" 200 OK
INFO:     127.0.0.1:51836 - "GET /static-files/static/css/main.280d6c90.css HTTP/1.1" 200 OK
INFO:     127.0.0.1:51835 - "GET /static-files/TelemetryLogger.telemetry-worker.706844bde809e5593e6c.worker.js HTTP/1.1" 200 OK
INFO:     127.0.0.1:51836 - "GET /static-files/static/js/5759.45405231.chunk.js HTTP/1.1" 200 OK
INFO:     127.0.0.1:51835 - "GET /static-files/static/js/3617.10568100.chunk.js HTTP/1.1" 200 OK
INFO:     127.0.0.1:51837 - "GET /static-files/static/css/8516.26533251.chunk.css HTTP/1.1" 200 OK
INFO:     127.0.0.1:51838 - "GET /static-files/static/js/8516.2c6c0e64.chunk.js HTTP/1.1" 200 OK
INFO:     127.0.0.1:51838 - "GET /static-files/manifest.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:51835 - "GET /static-files/favicon.ico HTTP/1.1" 200 OK
2025/12/31 23:46:02 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2025/12/31 23:46:02 INFO mlflow.store.db.utils: Updating database tables
2025/12/31 23:46:02 INFO alembic.runtime.migration: Context impl SQLiteImpl.
2025/12/31 23:46:02 INFO alembic.runtime.migration: Will assume non-transactional DDL.
2025/12/31 23:46:02 INFO alembic.runtime.migration: Context impl SQLiteImpl.
2025/12/31 23:46:02 INFO alembic.runtime.migration: Will assume non-transactional DDL.
INFO:     127.0.0.1:51835 - "GET /ajax-api/2.0/mlflow/experiments/search?max_results=5&order_by=last_update_time+DESC HTTP/1.1" 200 OK
INFO:     127.0.0.1:51836 - "GET /ajax-api/3.0/mlflow/ui-telemetry HTTP/1.1" 200 OK
```

6. System Architecture

6.1 High-Level Architecture Diagram



6.2 Component-Wise Explanation

Layers	Purpose and steps
Data Layer	**Purpose:** Reliable ingestion & preparation of high-quality data. • **Raw Data Sources** Structured and labeled datasets from CSV files, SQL databases, and object storage (e.g., S3). • **Data Ingestion** Connects to batch or streaming sources and loads data into the processing layer. • **EDA & Preprocessing** Performs data validation, cleaning, normalization, transformation, and feature engineering to create model-ready datasets.

Layers	Purpose and steps
Training & Experimentation Layer	**Purpose:** Model development, experimentation, and reproducibility. • **Model Training Pipeline** Trains multiple ML models (e.g., Logistic Regression, Random Forest) and evaluates them using standard performance metrics. • **Model Validation** Ensures trained models meet accuracy, stability, and business performance thresholds. • **MLflow Experiment Tracking** Centralized governance for: ◦ Hyperparameters ◦ Metrics ◦ Training runs ◦ Model versions and lineage • **Model Artifacts Store** Versioned, serialized model files used as deployment candidates.
Packaging & Serving Layer	**Purpose:** Consistent, reproducible model deployment. • **Docker Image** Packages the trained model, FastAPI inference code, and dependencies into a portable container. • **Model Serving API** REST-based inference service built using FastAPI to support real-time predictions.
CI/CD Automation (GitHub Actions)	**Purpose:** Fully automated, gated ML delivery pipeline. • **Code Quality & Tests** Executes linting and unit tests on every push and pull request. • **Build & Publish** Builds Docker images and pushes them to GitHub Container Registry (GHCR). • **Deploy & Validate** Automatically deploys containers, runs health checks, and performs inference smoke tests.

Layers	Purpose and steps
Observability & Monitoring Layer	**Purpose:** Production reliability, performance tracking, and continuous improvement. • **Monitoring (Prometheus)** Collects runtime metrics such as: ◦ API latency ◦ Request throughput ◦ Model health • **Logging & Diagnostics** Captures application logs for debugging, SLA monitoring, and auditability. • **Feedback Loop** Monitoring insights feed back into retraining and pipeline improvements.
Cross-Cutting Capabilities	• **Reproducibility** – Versioned data, models, and code • **Governance** – Experiment tracking and model registry • **Scalability** – Containerized serving and automation • **Automation** – CI/CD-driven lifecycle management

7. CI/CD and Deployment Workflow

7.1 CI/CD Tool

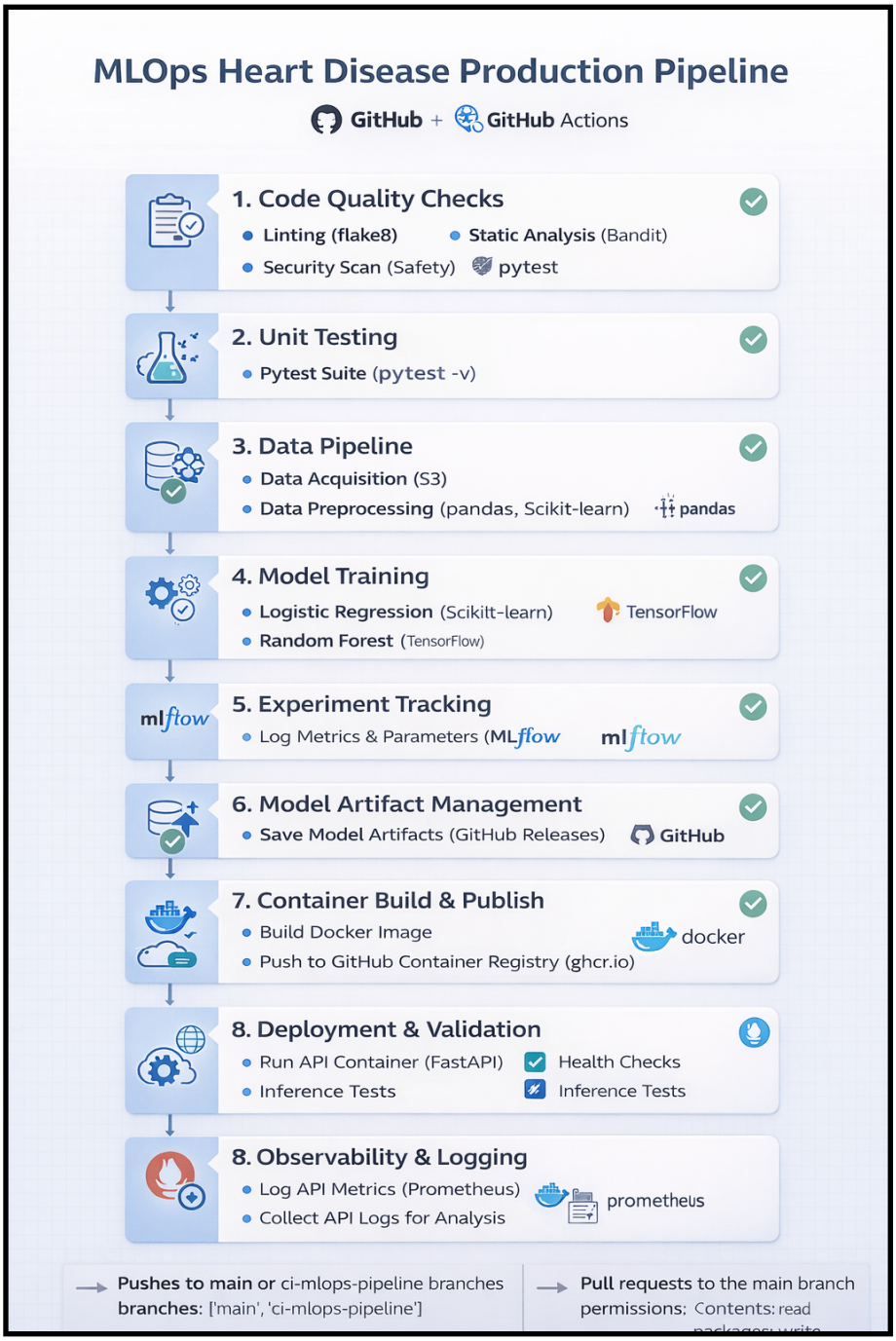
GitHub Actions

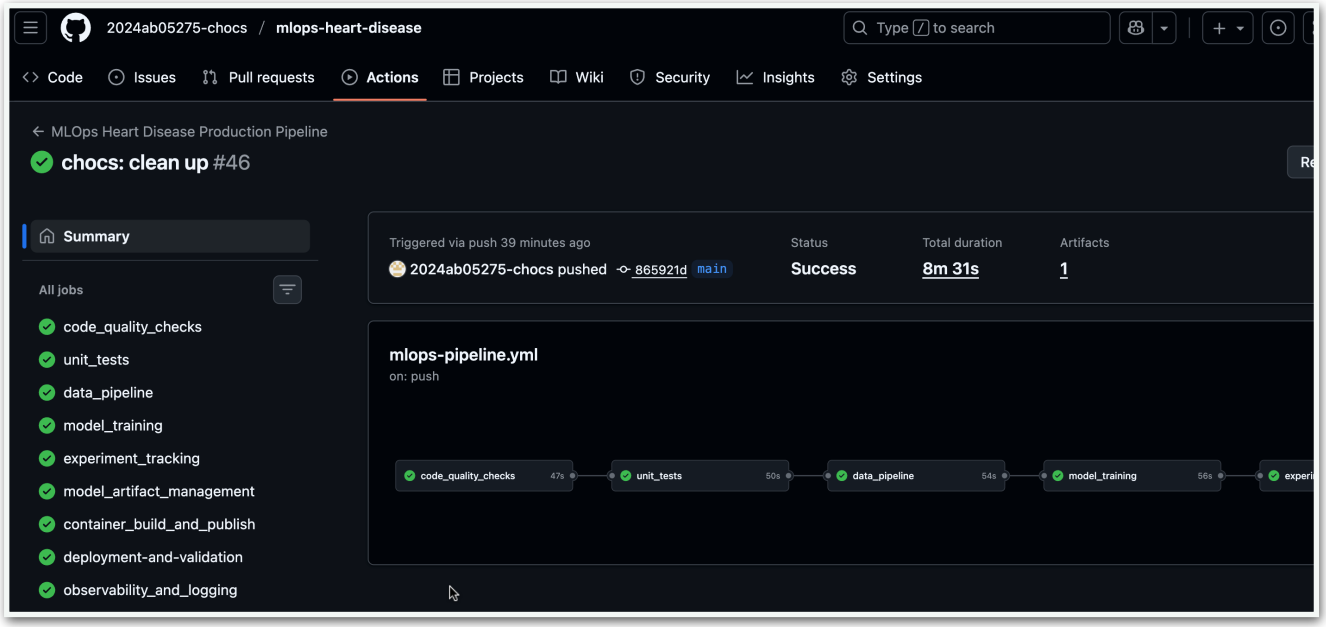
- **GitHub Actions** automates CI/CD by orchestrating linting, testing, training, containerization, deployment, and validation workflows.
- It ensures reliable MLOps pipelines through gated job dependencies, automated testing, Docker builds, and production health checks.

7.2 Pipeline Stages

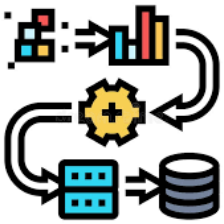
Sample Pipeline:

- https://github.com/2024ab05275-chocs/mlops-heart-disease/actions/runs/20672088742





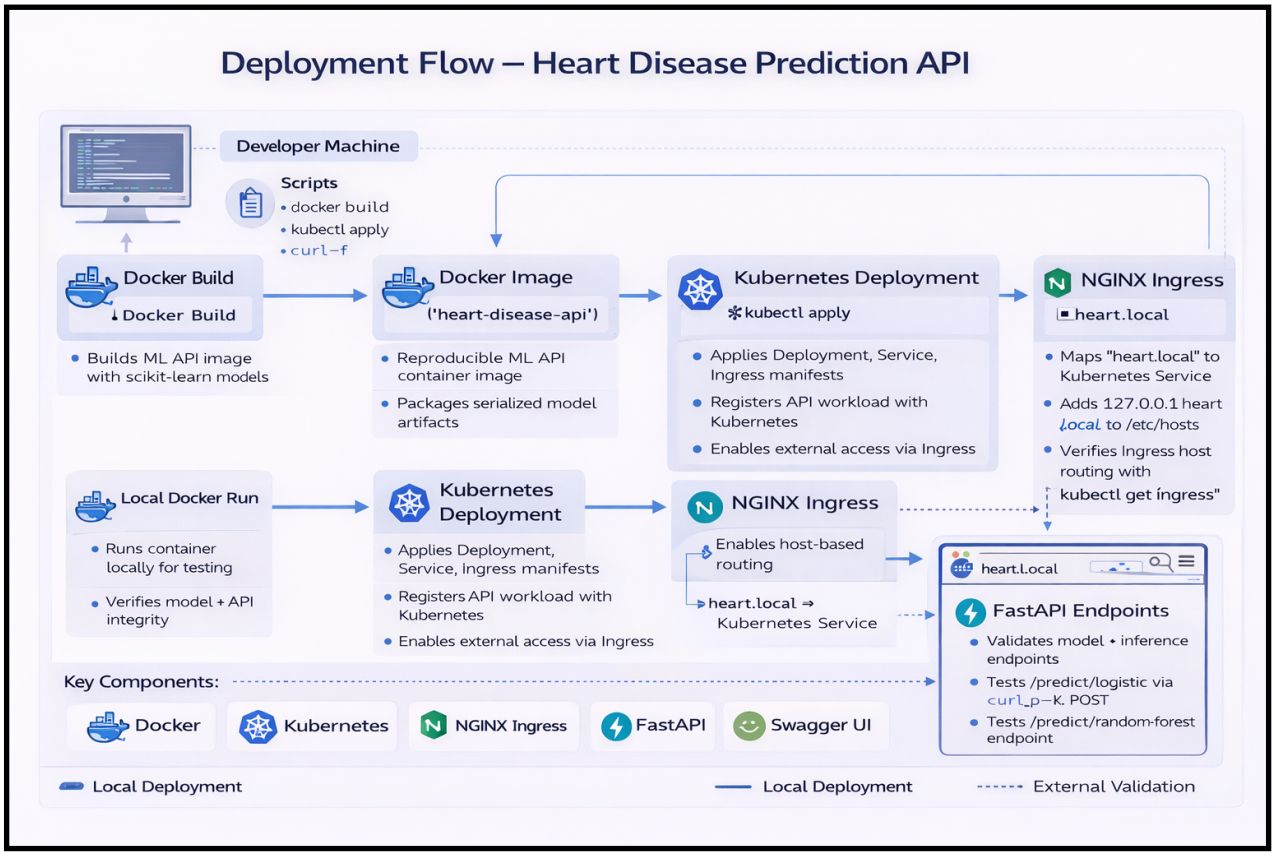
- **Code Quality Checks** – Runs linting and static analysis to enforce coding standards and catch issues early.
- **Unit Testing** – Executes automated pytest suites to validate core application logic.
- **Data Pipeline** – Performs data ingestion and preprocessing to prepare clean, model-ready datasets.

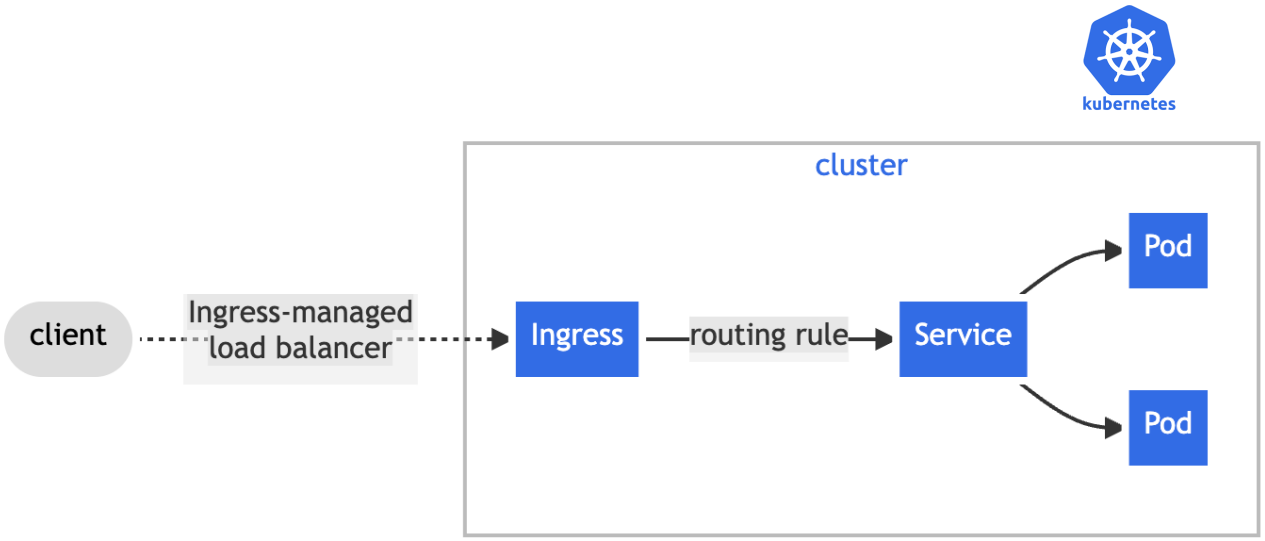


- **Model Training** – Trains machine learning models (Logistic Regression and Random Forest) on processed data.
- **Experiment Tracking** – Logs model parameters, metrics, and results for reproducibility and comparison.
- **Model Artifact Management** – Stores trained models and related artifacts in a versioned and traceable manner.
- **Container Build & Publish** – Builds Docker images and pushes them to the GitHub Container Registry.
- **Deployment & Validation** – Deploys the API container and validates it using health and inference tests.

- **Observability & Logging** – Collects runtime metrics and application logs for monitoring and diagnostics.

8. Deployment Workflow





8.1 Deployment Overview

- This deployment sets up a **containerized ML inference API** on a **local Kubernetes cluster** using **Docker Desktop + NGINX Ingress**.
- The pipeline ensures **clean rebuilds**, **Kubernetes rollout**, **Ingress-based access**, and **automated endpoint validation**.

8.2 Deployment Steps Followed

🔷 Step 0: Prerequisite Validation

Ensures the local system is ready for deployment.

- Docker CLI installed and running
- Kubernetes cluster enabled (Docker Desktop)
- kubectl available and connected
- NGINX Ingress Controller installed

✔ Prevents partial or broken deployments.

🔷 Step 1: Docker Cleanup

Ensures a clean runtime environment.

- Stops any running container with the same name
- Removes old containers to avoid port or image conflicts

🔷 Step 2: Build & Run Docker Image

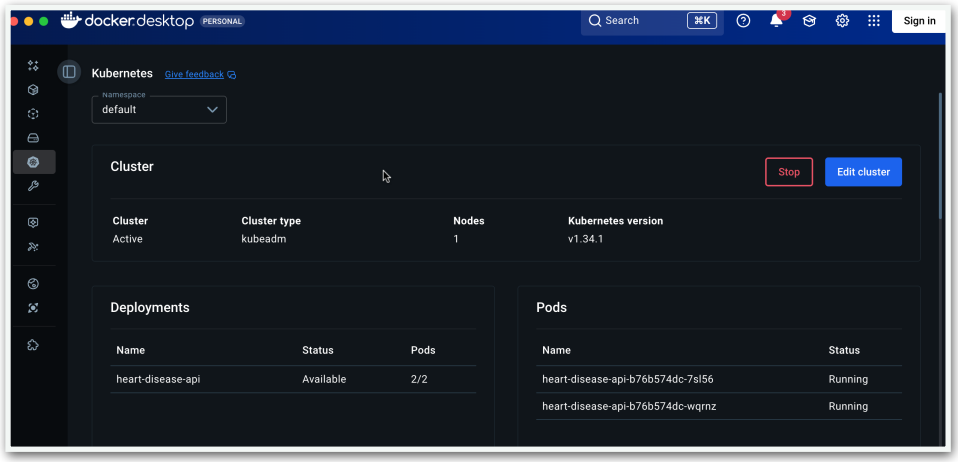
Packages the ML API into a container.

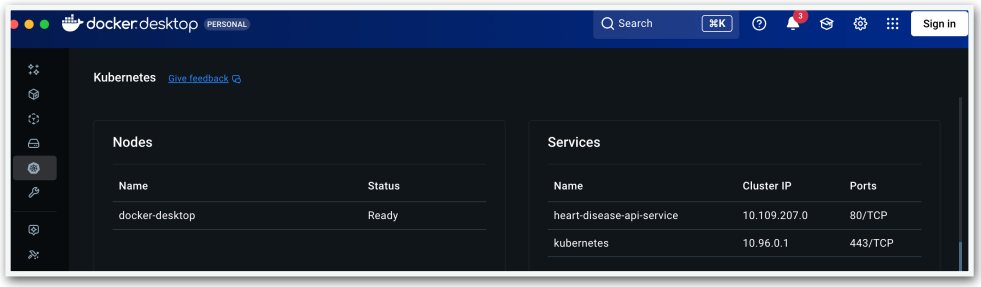
- **Builds Docker image** containing:
  - Trained ML models
  - FastAPI application
  - Dependencies
- Runs container locally to verify image integrity

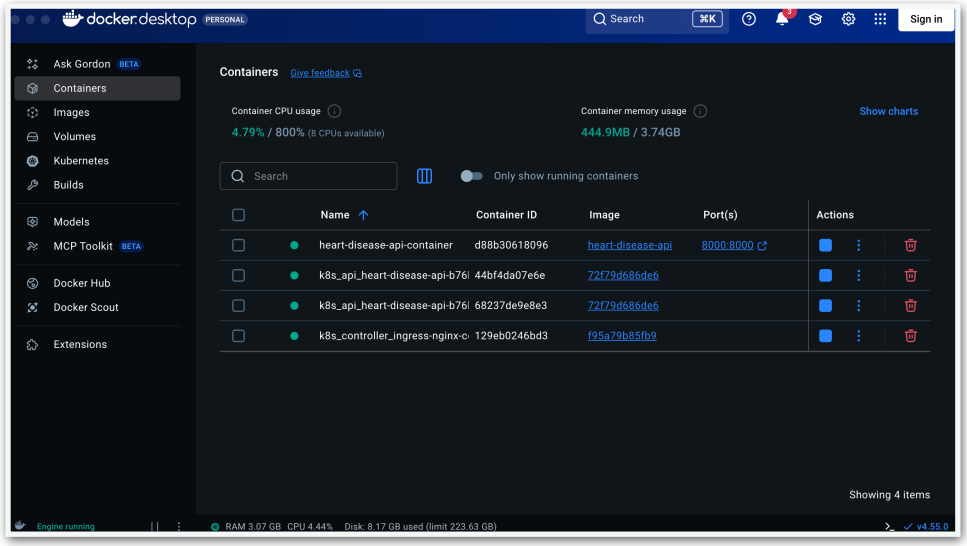
🔷 Step 3: Apply Kubernetes Manifests

Deploys application to **Kubernetes**.

- Applies Deployment, Service, and Ingress manifests from k8s/







- Registers the API as a Kubernetes workload

🔷 Step 4: Pod Restart & Readiness Check

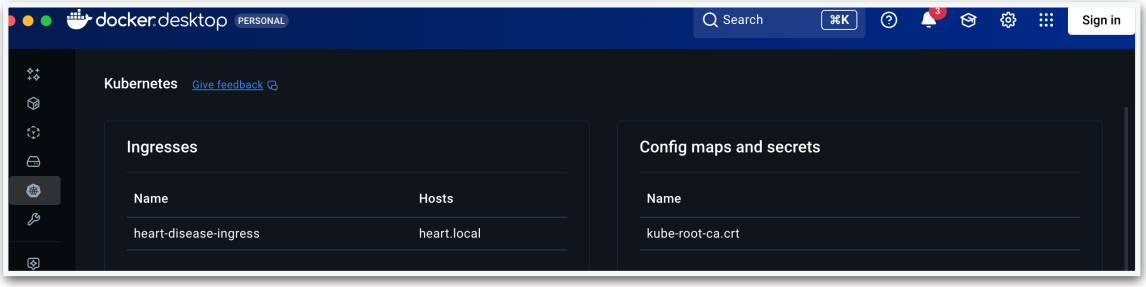
Ensures fresh deployment is live.

- Deletes old pods
- Waits for new pods to reach **Ready** state
- Guarantees zero stale containers

🔷 Step 5: Ingress Host Configuration

Enables domain-based access.

- Maps **heart.local → 127.0.0.1**
- Allows browser and curl access via hostname



🔷 Step 6: Ingress Verification

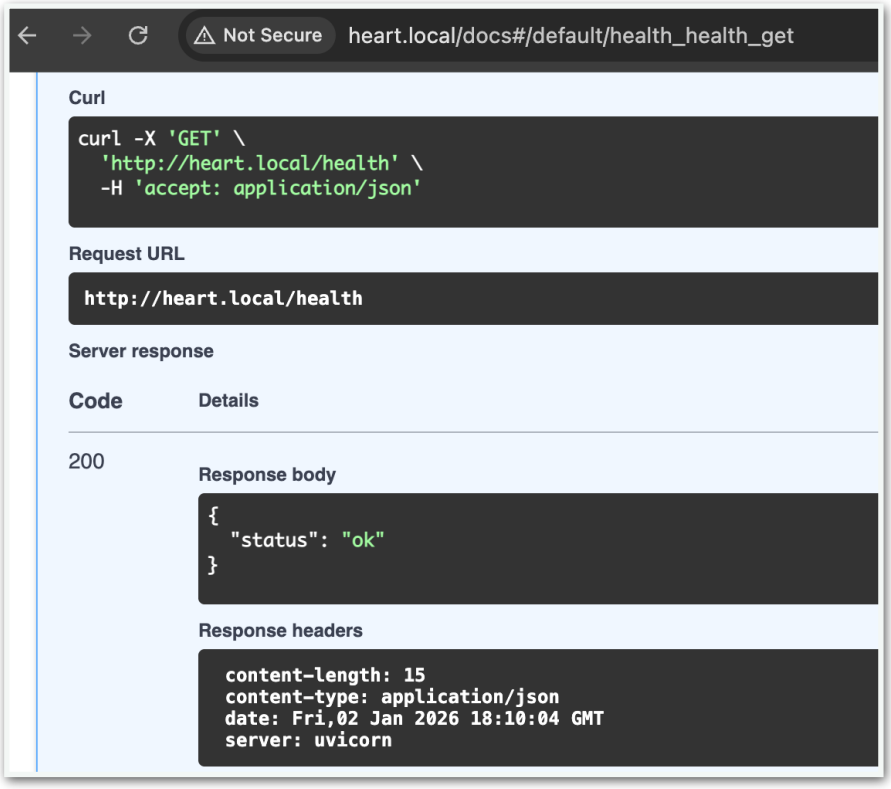
Validates routing configuration.

- Confirms Ingress resource is created
- Ensures correct host and service mapping

🔷 Step 7: Health Check Validation

Confirms application liveness.

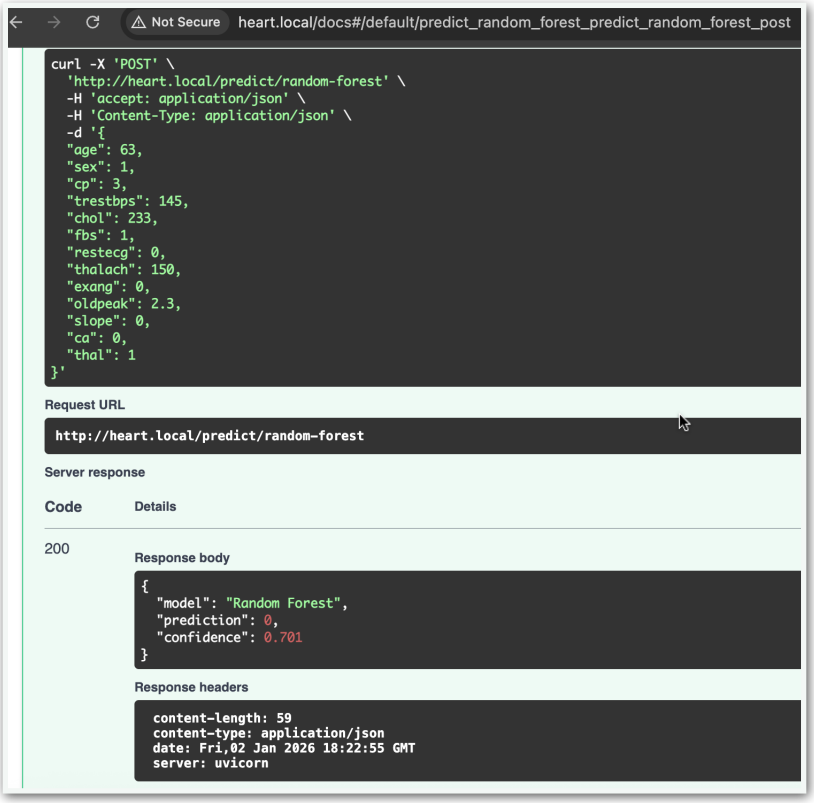
- Calls http://heart.local/health endpoint
- Verifies API startup success

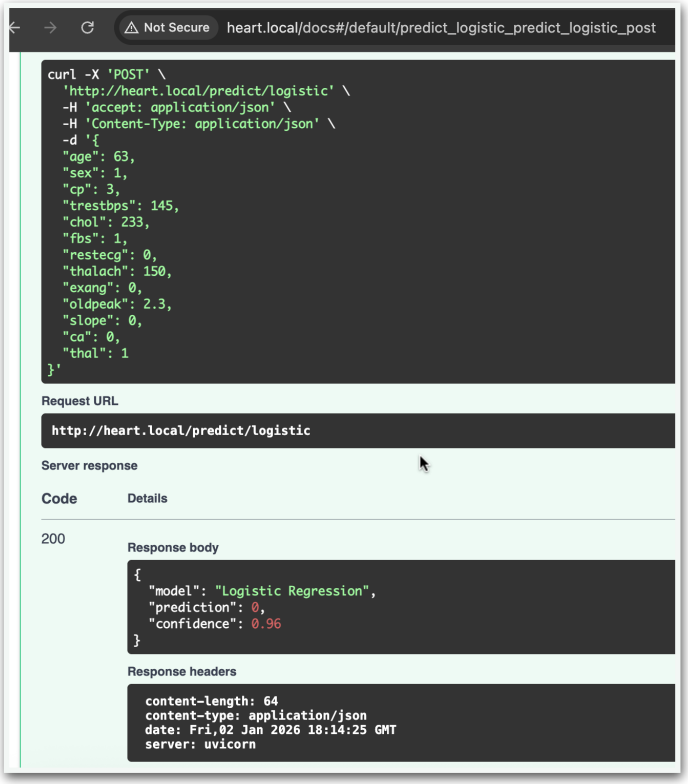


🔷 Step 8 & 9: Model Inference Validation

Validates ML endpoints.

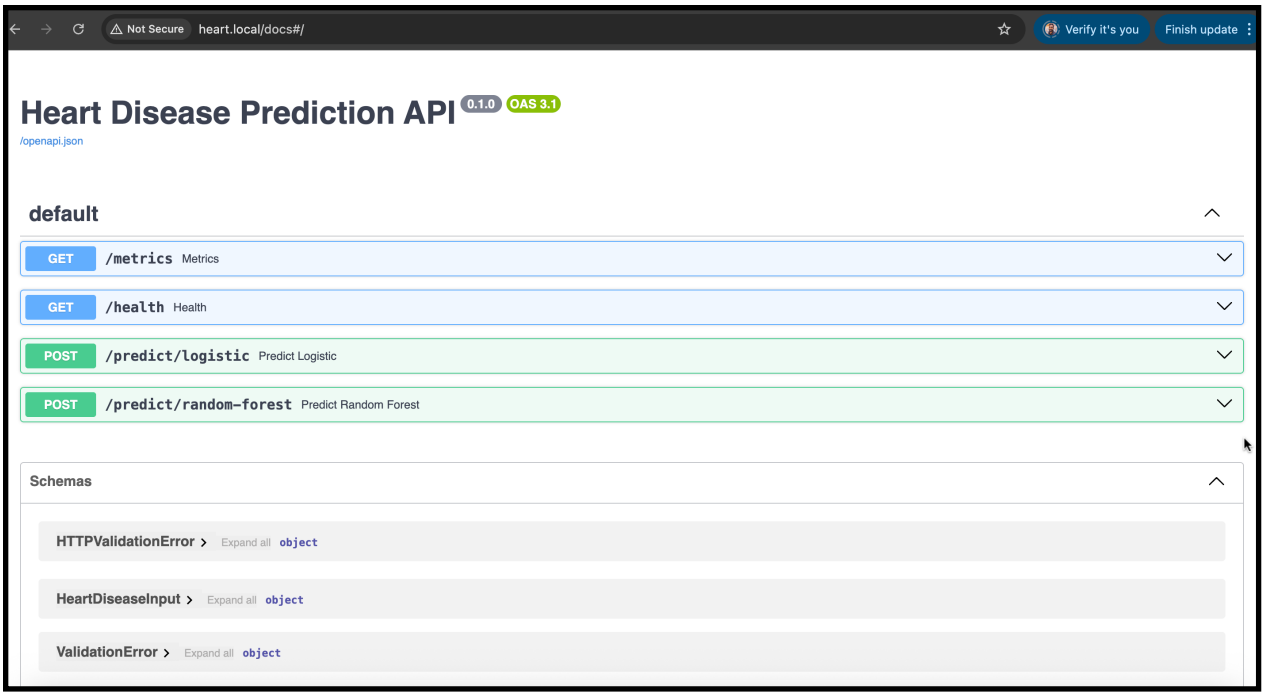
- Tests **Logistic Regression** prediction endpoint
  http://heart.local/predict/logistic
- Tests **Random Forest** prediction endpoint
  http://heart.local/predict/random-forest





🔷 Step 10: API Access Confirmation

- Swagger UI available at: **http://heart.local/docs**


Not Secure
heart.local/docs#/
Verify it's you
Finish update
Heart Disease Prediction API
0.1.0
OAS 3.1
/openapi.json
default
GET /metrics Metrics
GET /health Health
POST /predict/logistic Predict Logistic
POST /predict/random-forest Predict Random Forest
Schemas
HTTPValidationError Expand all object
HeartDiseaseInput Expand all object
ValidationError Expand all object